



Facultad de Ingeniería y Tecnología
Departamento de Ciencias Exactas y Naturales
Cálculo Aplicado
Optimización Numérica en una dimensión

1. Introducción

La optimización es una herramienta fundamental en matemática aplicada, ya que permite resolver problemas formulados como la búsqueda de valores que minimizan o maximizan una función. Este tipo de problemas aparece en contextos como sistemas físicos, ciencia de datos y la vida misma.

En este trabajo se busca encontrar de forma aproximada los puntos donde una función alcance un máximo o mínimo. Dado que en muchos casos no es posible resolver este problema analíticamente, utilizaremos métodos iterativos para aproximar la solución.

En particular, implementaremos y analizaremos distintos métodos clásicos en una dimensión, como el método de bisección, el método de Newton y el descenso por gradiente. Estos métodos difieren en la información que utilizan y en su comportamiento, por lo que resulta relevante comparar su desempeño en distintas situaciones.

Para verificar su funcionamiento, primero los aplicaremos sobre funciones conocidas. Luego se aplicarán en funciones más complejas y situaciones relacionadas a Inteligencia Artificial (simplificado a una variable).

El objetivo del trabajo es comprender cómo estos métodos permiten aproximar soluciones, así como analizar sus ventajas, limitaciones y sensibilidad a las condiciones iniciales y a los parámetros del algoritmo.

2. Implementación de métodos

En esta sección se implementarán distintos métodos clásicos de optimización en una dimensión. Para ello, se provee junto con la tarea un conjunto de abstracciones de funciones, que permiten evaluar una función y sus derivadas. Los métodos deberán implementarse utilizando esta interfaz.

El objetivo de esta parte es verificar que las implementaciones son correctas y producen resultados coherentes. No se requiere, en esta sección, un análisis detallado del desempeño de los métodos.

2.1. Métodos a implementar

Implementar los siguientes métodos:

- Método de bisección
- Método de Newton
- Descenso por gradiente (clásico)

Cada implementación deberá:

- Recibir las condiciones iniciales apropiadas para el método
- Incluir un criterio de parada (por ejemplo, tolerancia o número máximo de iteraciones)
- Devolver el punto obtenido y la cantidad de iteraciones realizadas

2.2. Verificación de funcionamiento

Para verificar que los métodos fueron implementados correctamente, utilizarlos sobre funciones simples donde el extremo pueda identificarse fácilmente.

Se sugiere trabajar al menos con las siguientes funciones:

- $f(x) = x^2$
- $f(x) = (x + 1/2)^3 - (x + 1/2)$
- $f(x) = \cos(x)$

Para cada método y cada función:

- Ejecutar el algoritmo con al menos una condición inicial razonable
- Verificar que el resultado obtenido sea consistente con el extremo esperado

No es necesario realizar un estudio comparativo entre métodos en esta etapa, sino únicamente comprobar que las implementaciones funcionan correctamente y producen resultados coherentes.

2.3. Preguntas

- ¿Pueden estos métodos diferenciar un máximo de un mínimo?
- ¿Qué ventajas y desventajas encuentran de cada uno?

3. Aplicación a una función no trivial

En esta sección se utilizarán los métodos implementados sobre una función más compleja, con el objetivo de observar su comportamiento frente a múltiples extremos locales.

3.1. Función de estudio

Considerar la siguiente función:

$$f(x) = x^2 + 2 \sin(3x)$$

3.2. Experimentos

- Grafique la función $f(x)$ en un intervalo adecuado (por ejemplo, $[-3, 5]$)
- Analice qué sucederá en las siguientes condiciones iniciales. ¿Qué predice que sucederá en cada una?
 - Para bisección: intervalos $[-2, 1]$, $[-1, 0]$, $[1, 2]$
 - Para Newton: $x_0 \in \{-2, -1, 4\}$
 - Para descenso por gradiente: $x_0 \in \{-3, 0.585, 4\}$
- Ejecute los métodos para cada condición inicial. Genere una tabla con el método, su predicción, el resultado de la aproximación y la cantidad de iteraciones.
- Para descenso por gradiente, pruebe con varios **learning rates** y discuta diferencias.
- BONUS: Modifique los algoritmos para devolver la sucesión de aproximaciones y graficar los puntos sobre la función para observar la evolución del modelo. Utilice un código de colores para ver el orden de los puntos.

3.3. Preguntas

- ¿Los métodos funcionaron como esperaba para las condiciones iniciales?
- ¿Cómo utilizaría descenso por gradiente para obtener máximos en vez de mínimos? (sin cambiar el algoritmo). ¿Hay riesgo de que no converja la búsqueda de máximos para alguna de las condiciones iniciales utilizadas?

4. Ajuste de datos mediante optimización

En esta sección se aplicarán las herramientas desarrolladas a un problema de ajuste de modelos a datos. Se proveerán distintos conjuntos de datos de la forma (x_i, y_i) , generados a partir de modelos conocidos con la adición de ruido. En todos los casos, se considerarán modelos parametrizados por un único parámetro $w \in \mathbb{R}$.

Los modelos que deberán ajustar son:

- $\hat{y}(x; w) = wx$
- $\hat{y}(x; w) = e^{wx}$
- $\hat{y}(x; w) = \sin(wx)$

4.1. Función objetivo

Para cada dataset, se define la función de pérdida:

$$L(w) = \frac{1}{N} \sum_{i=1}^N (\hat{y}(x_i; w) - y_i)^2$$

El objetivo es encontrar el valor de w que minimiza $L(w)$ utilizando descenso por gradiente.

4.2. Experimentos

Para cada uno de los modelos y su correspondiente dataset:

- Grafique la función $L(w)$ para cada uno de los datasets. ¿Qué comportamiento observa? ¿Algún otro método podría funcionar para obtener el resultado esperado?
- Aplique descenso por gradiente para obtener el w óptimo para algún **learning rate**. ¿Converge al resultado esperado?
- Graficar el conjunto de datos original y el modelo ajustado.

4.3. Preguntas

- ¿Cómo cambia la forma de $L(w)$ según el modelo?
- ¿Se observa un único mínimo en todos los casos?
- ¿Qué diferencias se observan en la convergencia del método?

5. BONUS:

5.1. Descenso por gradiente estocástico (SGD)

El descenso por gradiente estocástico (SGD) consiste en aproximar la derivada de la función objetivo utilizando solo una parte de la información total.

Este método puede aplicarse cuando la función objetivo puede escribirse como suma de términos, es decir:

$$L(w) = \frac{1}{N} \sum_{i=1}^N \ell_i(w)$$

En este caso, en lugar de utilizar la derivada completa $L'(w)$, se utiliza la derivada de un único término $\ell_i(w)$ (o de un subconjunto de términos).

Experimentos

- Implementar descenso por gradiente estocástico para los problemas de ajuste de datos
- En cada iteración, seleccionar un índice i y actualizar utilizando $\ell'_i(w)$
- Comparar cualitativamente su comportamiento con el descenso por gradiente estándar

Preguntas

- ¿Qué diferencias se observan en la convergencia?
- ¿El método converge exactamente o fluctúa alrededor del mínimo?

5.2. Descenso por gradiente con learning rate variable

Otra variante consiste en utilizar un learning rate que cambie a lo largo de las iteraciones. Implemente una de estas variantes.

Experimentos

- Optimice la función $f(x) = x^2 + 2 \sin(3x)$ para las condiciones iniciales del descenso por gradiente clásico.

Preguntas

- ¿Qué efecto tiene el learning rate variable en la convergencia?
- ¿Permite evitar oscilaciones o mejorar la estabilidad?

5.3. Formulación producto de exponenciales vs suma

En el caso del modelo lineal $\hat{y}(x; w) = wx$, considerar la siguiente función de pérdida:

$$\tilde{L}(w) = \prod_{i=1}^N e^{-(wx_i - y_i)^2}$$

Observando su logaritmo:

$$\log \left(\prod_{i=1}^N e^{-(wx_i - y_i)^2} \right) = - \sum_{i=1}^N (wx_i - y_i)^2$$

se observa que maximizar $\tilde{L}(w)$ es equivalente a minimizar $L(w)$.

Experimentos

- Optimice $\tilde{L}(w)$ con descenso por gradiente.
- Comparar la cantidad de iteraciones necesarias, y/o la utilidad de esta formulación alternativa como productoria.

Preguntas

- ¿Se obtiene el mismo valor de w en ambas formulaciones?
- ¿Cuál de las dos versiones converge más rápido? ¿Por qué?
- ¿Se observan problemas numéricos en la optimización de $\tilde{L}(w)$?
- ¿Cómo se compara la magnitud de los gradientes en cada caso?